

Hover v0.9.0 Language Manual

A Mixed-Systems Co-Simulation Language

Eray Mercan

Contents

1 Overview 1

2 File Structure & Imports 1

2.1 The `import` Directive . . . 1

3 Simulation Directives **1**
3.1 .tran — Transient Analysis . . . 1
3.2 .save — Signal Logging . . . 2
3.2.1 Logging Branch Currents . . . 2
3.3 .zcd — Zero-Crossing Detection . . . 3
3.4 Time Breakpoints . . . 3
3.5 .op — DC Operating Point . . . 4
3.6 .ac — Small-Signal AC Analysis . . . 5
3.7 .solver — Solver Type . . . 6

4 Data Types **9**
4.1 Structural Types . . . 9
4.2 Floating-Point Types . . . 9
4.3 Integer Types . . . 9
4.4 Arrays . . . 9
4.4.1 Multidimensional Arrays . . . 10
4.5 Pointers . . . 10
4.5.1 Address-of and Dereference . . . 10
4.5.2 Indexing . . . 11
4.5.3 Pointers as Function Parameters . . . 11
4.5.4 Pointers as State . . . 11

5 Number Literals & Suffixes **11**

6 Operators **12**
6.1 Arithmetic . . . 12
6.2 Bitwise & Integer Operators . . . 12
6.3 Comparison (return 1.0 or 0.0) . . . 12
6.4 Logical . . . 13
6.5 Precedence (highest to lowest) . . . 13

7 Variables & State **13**
7.1 Local Variable . . . 13
7.2 State Variable . . . 13

8 Control Flow **14**
8.1 If / Else If / Else . . . 14
8.2 While Loop . . . 14

9 Functions **14**
9.1 Declaration . . . 14
9.2 Call . . . 15

10 Modules **15**
10.1 Domains . . . 15
10.2 Domain Restrictions . . . 16
10.3 Why Two Computational Domains . . . 16
10.4 Bracket Convention . . . 17
10.5 Declaration . . . 17
10.6 Instantiation . . . 18
10.6.1 Named Parameters and Defaults . . . 18

10.7 Execution Order . . . 18

11 C/C++ Integration (FFI) 19
11.1 The `importc` Directive . . . 19
11.2 `extern func` Declarations . . . 19
11.3 Putting it Together . . . 19
11.4 Handling Structs and Complex Types . . . 20
11.5 The Opaque Pointer Pattern . . . 20

12 Physical Primitives 21
12.1 Named Elements . . . 22
12.2 Current-Controlled Sources (CCCS / CCVS) . . . 22
12.2.1 The 0 V Ammeter Idiom . . . 23
12.2.2 Terminal Conventions . . . 23
12.2.3 Worked Example . . . 23
12.3 Driven Sources (Logic-controlled) . . . 24
12.4 Fixed Sources (Compile-time value) . . . 24

13 Built-in Intrinsics 24
13.1 Reading Currents — `I()` . . . 24
13.1.1 Which Elements Have a Current . . . 25
13.1.2 Sign Convention . . . 25
13.2 nr_prev() . . . 25
13.2.1 nr_prev() — Supported Forms . . . 26
13.2.2 nr_prev() of a variable — the limiter's own anchor . . . 27
13.3 nodiff() . . . 28

14 Special Keywords 29

15 Simulation Run Order 29

16 Hovercraft — Library Mode 30
16.1 Compiler Options . . . 30
16.2 What Becomes Part of the ABI . . . 30
16.2.1 Params vs. Inputs — an Important Distinction . . . 31
16.3 Driving the Simulation . . . 31
16.4 Reading Results . . . 32
16.5 Return Codes . . . 32
16.6 Self-Description — `HVR_manifest()` . . . 32
16.7 Instance Model . . . 33
16.8 Caveat — What a `<>` Param Can Reach . . . 33

17 Full Example 34

1. Overview

Hover is a co-simulation language that combines **logic** (software-style computation) with **physical** (circuit) descriptions in a single source file. The compiler flattens a module hierarchy into a netlist, solves the circuit using Modified Nodal Analysis (MNA) at each timestep, and executes the logic blocks between steps.

2. File Structure & Imports

Hover supports modularizing your code across multiple source files. You can bring external components, such as device models from the standard library or custom sub-circuits, into your current file using the `import` directive.

2.1 The `import` Directive

Imports must be declared at the top of the file, outside of any module or function definitions. The target file is specified as a string literal, representing a path relative to the current file's directory or the compiler's include path.

```
// Import standard math functions from the standard library
import </math/math.hvr>;

// Import a standard BJT device model from exact path
import "./standard_library/semiconductors/npn_bjt.hvr";

module main<>()[] {
    // NPN from the imported file is now available in the global
       scope
    module q1 = NPN<1e-14, 100, 0.026>()[base, coll, emit];
}
```

3. Simulation Directives

Directives appear at the top level, outside any module, and start with .

3.1 .tran — Transient Analysis

```
.tran(start, stop, t_step);
```

Argument	Description
`start`	Start time (currently always 0)
`stop`	End time
`t_step`	The **largest** step the simulation may take. On a fixed-step solver every step is exactly `t_step`; on a variable-step solver it is the ceiling *max_dt*, and the controller may go below it but never above.

```
.tran(0, 40m, 1u); // 0 to 40 ms, steps of at most 1 us
```

One step-size argument, one meaning. `.tran` takes exactly three arguments. There is no separate `max_step`, and there is no minimum-step argument.

You do not choose the starting step. The solver picks it with SPICE's first-timestep rule, min(`t_step`, `stop`/50)/10, and grows from there. If that opening step will not converge the solver halves it and retries — repeatedly, and likewise for the DC operating point, which retries with progressively more damping before it will report failure. So a `t_step` that is slightly too coarse for the first microsecond of a circuit costs a few retries, not a failed run.

There is no lower bound on dt, by design. Enforcing one was implemented and reverted: it broke both BJT amplifier examples at $t = 0$ even when set as low as 1 ns, because a bipolar cold start genuinely needs picosecond steps for a few steps before it settles. Any bound loose enough to allow that is too loose to bound anything. Runaway back-off is instead limited internally, relative to where the back-off began, which needs no number from the deck.

Migrating from the four-argument form. `.tran(start, stop, tstep, max_step)` was accepted until 2026-08-11, where `tstep` was the *initial* step rather than a minimum. Drop the third argument and keep the fourth:

```
.tran(0, 50m, 1u, 50u);    // old
.tran(0, 50m, 50u);        // new -- identical ceiling
```

A four-argument `.tran` is a compile error carrying this same instruction, never a silent reinterpretation. The old `tstep` is not worth preserving: on a variable-step solver it only set where the run started, and within roughly 35 steps every adaptive solver had grown to `max_step` anyway. Its one lasting effect was the *phase* of the time grid — which is why `optocoupler.hvr` used to fail at `tstep = 0.1u` and complete at `1u` with identical step sizes throughout, purely because one alignment landed a step across an LED's turn-off and the other did not.

Raising `t_step` remains a useful first response to a circuit that will not converge: the companion conductance of every capacitor scales as C/dt, so very small steps make the matrix progressively harder to solve rather than easier.

3.2 .save — Signal Logging

```
.save(signal1, signal2, ...);
```

Signals are referenced by their fully-qualified mangled name using dot notation.

```
.save(main.lc_out, main.generator.carrierWave);
```

- If the name matches an MNA node it is read from the circuit solution vector.
- If it matches a *named circuit element* that owns a branch row, its **current** is logged — see below.
- Otherwise it is read from the logic values map (VM signal).

Results are exported to `simulation_output.csv`.

3.2.1 Logging Branch Currents

A `.save()` argument may also name a circuit element, in which case the *current through it* is logged rather than a voltage. Two spellings are accepted and mean exactly the same thing:

```
.save(main.vsense);        // bare element name
.save(I(main.vsense));     // explicit, and clearer at a glance
```

Either way the CSV column is headed `I(main.vsense)`, so a current column is never confusable with a voltage column. Element names and net names are guaranteed disjoint by the compiler, so the bare form is unambiguous.

Only elements that own an MNA branch row can be logged this way — `voltage_source`, `L`, `VCVS` and `CCVS` (see Section 13.1). Naming a resistor is a compile error that points you at the ammeter idiom rather than silently logging zeros.

Column order: node voltages and logic signals come first, in the order written; every branch-current column is appended after them.

3.3 .zcd — Zero-Crossing Detection

```
.zcd;
```

Logic (structural and digital modules) runs once at the start of each step and holds its outputs for the step. When a logic output jumps at an instant *inside* a step — a comparator flips, a PWM carrier crosses its reference, a friction torque reverses with the speed — the step would integrate the old value past it. `.zcd` finds that instant and ends the step there.

After a step has been solved, the logic is re-run at the step's start, end and middle, with the solution interpolated along the step, and everything the logic drives into the circuit — every driven source, and every logic value that analog code reads — is compared. A value that changes by more than 0.1 % of its size, and not smoothly, has jumped: the jump is bisected to 10^{-6} of the step, the step is re-solved to end just after it, and the next step restarts the integrator's history with a short step. Because the solution is interpolated, events caused by the circuit (a speed crossing zero) are found as well as events caused by `time`.

What .zcd cannot see: a value that is the same at both ends of the step. A pulse that begins and ends inside one step vanishes. Comparisons of `time` with a fixed threshold no longer depend on `.zcd` for this — they are time breakpoints (next section), with or without `.zcd`. What still relies on `.zcd`, and so still needs a maximum step shorter than the shortest pulse that matters:

- crossings of a nonlinear function of time with a level — `sin(w*time) > 0.5`, a triangle carrier against a reference (sine-triangle PWM), `time * f > n`;
- a time expression held in a variable before it is compared (`double ph = time * f; ... ph - floor(ph) < d`), or passed into a function parameter;
- phase accumulators and state machines (`ph = ph + f*dt; if (ph >= 1) ...`);
- thresholds that change within a step (computed from `dt`, from a user function, or in analog code from `V()`/`I()`);
- events caused by the solution rather than by `time`.

Fixed-step solvers: `euler_fixed` and `gauss_siedel` check before each step, along time only (the solution held), and shorten that one step; the next step is the full step again. `trapezoidal_fixed` has no event location.

When to use: any circuit where logic switches discretely — PWM comparators, relays, digital controllers, friction models.

3.4 Time Breakpoints

A comparison of `time` with a threshold is an event whose instant the deck states outright, as a SPICE `PULSE` source states its corners. The compiler finds these comparisons and the variable-step

solvers (`ndf2`, `bdf2`, `trapezoidal`, `euler_adaptive`) end a step exactly on each instant at which one flips — always, with or without `.zcd`. Nothing has to be declared:

```
digital module Pulse<double t0, double w, double amp>(output double i
    ) []
{
     i = 0.0;
     if (time >= t0 && time < t0 + w) { i = amp; }    // a 1 us pulse
         in a 100 ms step: exact
}
```

Recognised: S *op* X or X *op* S with *op* one of `>`, `>=`, `<`, `<=`, in digital, structural and analog modules and in functions they call, where S is

- `time`, `time + c`, `c + time` or `time - c`;
- a sawtooth of one of those: `fmod(u, T)`, `u % T` or `u - floor(u / T) * T` — pulse trains such as `fmod(time, T) < w` flip at kT and $kT + w$;

and the threshold X, `c` and `T` do not change within a step: literals, parameters, inputs, `state` variables of logic (so a dead-time module's `time - t_off >= t_dead`, with `t_off` a `state` set to `time` at turn-off, is exact), and extern math functions of these. Any other variable assigned from `time`, `dt` or a user function — or, in analog code, from `V()`, `I()`, `ddt()`, `idt()` — does not qualify, and that comparison is left to `.zcd`. So are `==`/`!=` and comparisons in clocked modules, which only sample at their ticks.

Exact landing. The step ends on the first floating-point time at which the comparison, evaluated exactly as written, changes its result: `time >= t0` at `t0` itself, `time > t0` at the next representable time after it. The next step's logic pass therefore sees the new value. On the step that ends on a breakpoint, analog code is shown the instant just before it (the last instant of the old value); the step that starts there sees the new one. A step that starts on a breakpoint restarts the integrator's history with a short step, as after a `.zcd` event.

Limits. A flip less than 10^{-9} of the maximum step after a step's start is not landed — the next sample sees it — and flips closer together than that are landed together, on the last of them; so no run can be driven to a vanishing step. A threshold inside the very first step of a run without `.op` (the bias step, at most a tenth of the maximum step) is not landed: that step integrates the logic as it stands at $t = 0$. To find the breakpoints in logic, the step's logic pass runs before the step size is chosen rather than after — logic cannot tell, since it reads t_n and the solution at t_n either way. Fixed-step solvers keep their grid — the step the deck chose is its sampling — and see no breakpoints. Decks without a recognised comparison run exactly as before.

3.5 .op — DC Operating Point

```
.op;
```

Instructs the VM to solve for the DC operating point before the transient begins. Without `.op`, all capacitors start uncharged and voltage sources snap to their values in a single linear solve — the circuit charges from zero initial conditions, producing a potentially long startup transient before reaching steady state.

With `.op`, the solver uses source stepping: the supply voltages and driven sources are scaled from 5 % to 100 % in twelve alpha steps. At each alpha level, up to 500 Newton-Raphson iterations are run with a 0.05 V per-iteration damping limit. The solution at $\alpha = 1.0$ is then relaxed to DC (pseudo-transient continuation: the step is grown geometrically up to 10^{12} s with the history anchored at the previous answer), so capacitors charge to their DC voltages and inductors carry

their DC currents. The result is committed as the initial condition for the transient, so the waveform begins at the settled operating point.

When to use: any circuit with a well-defined DC bias point — amplifiers, filters, motor drives. Omit `.op` only when the startup transient itself is the subject of the simulation.

3.6 .ac — Small-Signal AC Analysis

```
.ac(dec, 100, 1k, 1G);     // points per decade
.ac(oct, 20, 1k, 1G);      // points per octave
.ac(lin, 1000, 0, 1Meg);   // points in total
```

Solves the circuit linearised around its DC operating point at every frequency of the sweep: $(G + J + j\omega C)\, X = B_{ac}$, where J is the Jacobian of every nonlinear model at the operating point. The operating point is the full `.op` one (capacitors charged, inductors carrying DC) whether or not the deck says `.op`. Charges written with `ddt()` linearise automatically, so junction capacitances need no extra work.

Excitation is never in the directive: it is the `ac` and `ac_phase` (degrees) named parameters of the independent sources (Section 10.6.1). Both default to 0, so an unmarked source is a plain DC source in the sweep.

```
current_source<0, ac=1>()[gnd, die];      // 1 A into die: V(die) = Z(f)
voltage_source<2.5, ac=1>()[fb, gnd];     // 2.5 V bias + 1 V small signal
voltage_source<ac=1>(v_sig)[in, gnd];     // driven source + AC excitation
```

For a source driven from `()`, the expression's dependence on node voltages and currents enters the linearisation through J; its dependence on `time` is frozen at its $t = 0$ value (a `cos` source freezes at its peak, an `if (time > t0)` step before the step) and is *not* an excitation — only `ac=` is. `ac` and `ac_phase` are static: they may only appear inside `<...>`.

Output goes to `ac_output.csv`: a `Freq` column, then a `.re` and `.im` column for every `.saved` node and branch current (e.g. `main.die.re`, `main.die.im`). Logic signals have no small-signal value and are left out. A deck with both `.ac` and `.tran` runs the sweep first and the transient after it, unaffected; a deck with only `.ac` runs no transient. `.ac` is not available with `-hovercraft`.

3.7 .solver — Solver Type

```
.solver(solverName, reltol, vntol, max_iter, abstol);
```

Every argument after `solverName` is optional, and passing `0` for any of them means "use this solver's default" rather than "use zero". Arguments that the chosen solver does not support are ignored, as in SPICE's `.OPTIONS`.

Argument	Default	Applies to
`reltol`	`1e-3`	Relative tolerance, every row.
`vntol`	`1e-6` V	Absolute tolerance for **node voltages**.
`max_iter`	`100`	Newton iterations before the step is rejected.
`abstol`	`1e-12` A	Absolute tolerance for **branch currents**.

Change, 2026-08-10. Argument 2 was previously named `abstol` and was applied to every row of the solution vector. That vector holds node voltages in its first rows and branch currents in the rest, so one absolute tolerance could not be right for both: `1e-6` is a sensible microvolt on a node and a very loose microamp on a branch. Argument 2 keeps its position and now applies to voltages only (SPICE calls this `vntol`), and a fifth argument `abstol` was appended for currents, defaulting to SPICE's `1e-12` A. Existing decks are unaffected in their voltage behaviour; branch currents are now held to a tighter tolerance than before, which can change iteration counts.

Which solvers honour the split, 2026-08-11. `abstol` applies to `bdf2`, `ndf2`, `trapezoidal` and `euler_adaptive`, which share one convergence test and split the solution vector by row. On `gauss_siedel` and `trapezoidal_fixed` argument 5 is ignored and argument 2 still applies to every row, as everywhere before this change; on `euler_fixed`, which runs no Newton loop, no tolerance argument applies at all. Ignoring an argument the active method does not use is SPICE `.OPTIONS` behaviour and is never an error.

Solvers

Implicit solvers treat the circuit as a Differential-Algebraic system and solve $Gx = b$ at each timepoint, enforcing Kirchhoff's laws and voltage source constraints simultaneously. They are unconditionally stable for linear circuits. Nonlinear devices require an inner Newton-Raphson loop. Every HVR implicit solver uses **Modified Newton**: the Jacobian is reused across multiple inner iterations rather than recomputed every iteration, reducing cost relative to a full Newton scheme. Reuse policy differs by solver — some recompute once per timestep unconditionally, others carry a Jacobian across many timesteps and only refresh it once a convergence failure proves it stale — see each solver's entry below.

Per-iteration step damping also differs by solver. All use **diagonally-scaled trust-region** damping: each variable's proposed step is measured relative to its own current magnitude rather than against a single global voltage budget, so the same solver behaves correctly on a millivolt-scale junction and a kilovolt-scale bus within one circuit.

Solver	**Description**	**Simulink**	**SPICE**
`euler_fixed`	Backward Euler, fixed timestep, first-order, no tunable parameters. A single implicit solve per step with no error control. Use as a debugging baseline or when a known-simple, always-converges reference trace is needed.	`ode1`	`N/A`
`euler_adaptive`	Backward Euler with Modified Newton-Raphson, diagonally-scaled trust-region damping, and convergence-driven adaptive timestepping. Expands dt by $1.5\times$ when the inner loop converges quickly, contracts by $0.5\times$ on failure. General-purpose first-order solver when a fixed-order, fixed-step solver is too slow.	`ode15s` (1st order)	`N/A`
`gauss_siedel`	Backward Euler with fixed-point relaxation under-relaxation ($\omega = 0.05$). No Jacobian formed at all — structurally independent of every Jacobian-related failure mode the other solvers share. Converges slowly but predictably; useful as a diagnostic cross-check when a Newton-based solver's convergence behavior is in question, since a difference in outcome isolates the problem to the Jacobian/Newton machinery rather than the circuit itself. Not recommended for general use.	N/A	N/A
`trapezoidal`	Variable-step Crank-Nicolson with Modified Newton-Raphson and diagonally-scaled trust-region damping. A-stable. Falls back to Backward Euler for one step after a ZCD event or a rejected step, to suppress numerical ringing before resuming 2nd-order accuracy.	`ode23t`	`Trap`

Solver	**Description**	**Simulink**	**SPICE**
`trapezoidal_fixed`	The fixed-step counterpart to `trapezoidal`: same 2nd-order Crank-Nicolson formula and the same Modified Newton loop, but with diagonally-scaled trust-region damping and no adaptive step-size control at all. Intended as a foundation-testing tool — isolates whether a convergence failure comes from the core Newton/Jacobian mechanics or from the adaptive step-size layer on top of it.	`ode23t`	`Trap`
`bdf2`	2nd-order Backward Differentiation Formula (Gear's method) with Modified Newton-Raphson, diagonally-scaled trust-region damping, and adaptive timestepping. L-stable — actively damps spurious oscillations after switching events. Uses a Backward Euler primer on startup and after any step rejection.	`ode15s`	`Gear`
`ndf2`	Shampine & Reichelt's order-2 Numerical Differentiation Formula — the real algorithm underlying `ode15s` — with diagonally-scaled trust-region damping. Adds a κ-weighted correction term to plain BDF2, improving accuracy within the same A-stable order-2 regime (per the Dahlquist second barrier, no linear multistep method of order 3 or higher can be A-stable, so this solver deliberately does not extend past order 2). Starts at order 1 and promotes to order 2 once enough step history exists.	`ode15s`	`Gear` (NDF variant)

4. Data Types

Hover provides distinct data types to represent both physical electrical nets and software-style logic variables.

4.1 Structural Types

Structural types are used purely for circuit topology and cannot hold logic states or numerical values.

Type	Description	Example
`wire`	Electrical net (topological only)	`wire in, out;`

4.2 Floating-Point Types

Floating-point numbers represent the continuous domains of time, voltage, and current.

Type	Description	Example
`double`	64-bit floating point	`double x = 3.14;`
`float`	32-bit floating point	`float y = 1.0;`

Backend Execution Note: While `float` variables are executed as true 32-bit C++ floats locally inside `digital` module logic, the underlying MNA solver and global state maps operate exclusively on 64-bit precision. Therefore, `float` values are implicitly promoted to `double` when crossing into analog modules, driving physical sources, or being logged via `.save`. For all physics and solver-related calculations, `double` is highly recommended to prevent numerical instability during Newton-Raphson iterations.

4.3 Integer Types

Integer types are primarily used for discrete digital logic, state machines, loop counters, and bitwise operations.

Type	Description	Example
`int`	Signed 32-bit integer	`int n = -4;`
`unsigned int`	Unsigned 32-bit integer	`unsigned int k = 42;`

Assigning a floating-point value to an integer variable implicitly truncates the fractional portion. Furthermore, the `unsigned` keyword is strictly reserved for integer types; an `unsigned double` is invalid.

4.4 Arrays

Append `[size]` to any type declaration to create fixed-size arrays:

```
wire[8]   databus;
double[3] coeffs = {1.0, 2.5, 0.5};
int[4]    counts = 0;    // all elements initialised to 0
float[4]  gains  = 31u; // all elements initialised to 31e-6
```

Note: `wire` arrays cannot be initialised with a value literal, as they do not hold data.

4.4.1 Multidimensional Arrays

Append additional `[size]` suffixes to declare arrays of two or more dimensions. Dimensions are read outer-to-inner, matching C:

```
double[2][3] m      = {{1.0, 2.0, 3.0}, {4.0, 5.0, 6.0}};
int[2][2]    grid   = 0;     // every element initialised to 0
float[3][3]  kernel = 5u;    // every element initialised to 5e-6
```

Initialisation follows the same two forms as a single-dimension array, applied recursively at every nesting level:

- **Nested brace literals** — `{{...}, {...}}` — assign each inner list to the corresponding row. The nesting depth must match the number of declared dimensions.
- **Scalar-fill** — a single value broadcasts to *every* element across all dimensions, not just the outermost one. `int[2][2] grid = 0;` zero-fills all four elements, exactly like the one-dimensional case.

Indexing a multidimensional array peels off one dimension per subscript, left to right:

```
double[2][3] m    = {{1.0, 2.0, 3.0}, {4.0, 5.0, 6.0}};
double[3]    row0 = m[0];    // one subscript -- the first row
double       x    = m[1][2]; // two subscripts -- single element,
    value 6.0
```

When a multidimensional array is passed as a function argument, only its outermost dimension is allowed to decay; every inner dimension must remain a fixed, known size:

```
func double sumRow(double[3] row) {
  return row[0] + row[1] + row[2];
}
```

Note: as with one-dimensional arrays, `wire` arrays of any dimensionality cannot be initialised with a value literal.

4.5 Pointers

Append `*` to any numeric type declaration to create a pointer. Pointer stars stack the same way C's do — one `*` per level of indirection — and follow the same declaration grammar as arrays, so the two can be combined:

```
double*  p;       // pointer to a double
int**    pp;      // pointer to a pointer to an int
double*[4] parr;  // fixed-size array of 4 double pointers
```

A pointer declared without an initializer defaults to `null`, exactly like a scalar defaults to `0`.

4.5.1 Address-of and Dereference

`&` takes the address of a variable, producing a pointer to its type; `*` in prefix position dereferences a pointer, producing its pointee's type. These are the same two symbols used elsewhere as bitwise AND and multiplication — Hover disambiguates purely by position, exactly as C does: in front of an operand with nothing to its left, they are unary; between two operands, they are binary.

```
double x = 3.14;
double* p = &x;   // p now holds the address of x
double  y = *p;   // y == 3.14, read through the pointer
```

```
4
5 *p = 2.5;          // write through the pointer -- x is now 2.5
```

4.5.2 Indexing

A pointer can be subscripted with `[]` exactly like an array — `p[i]` is equivalent to `*(p + i)` in the underlying generated code, following normal C pointer/array duality:

```
1 double* p = &coeffs[0]; // coeffs from the Arrays example above
2 double  first = p[0];   // same as coeffs[0]
```

4.5.3 Pointers as Function Parameters

Unlike scalar parameters (which are copied), a pointer parameter is aliased: the function receives the same address the caller passed, so writes made through it inside the function are visible to the caller after it returns. This mirrors how array parameters behave (§4.4).

```
1 func void increment(double* p) {
2   *p = *p + 1;  // modifies the caller's variable directly
3 }
4
5 double x = 5.0;
6 increment(&x); // x is now 6.0
```

4.5.4 Pointers as State

Because a pointer is just an address-sized value, it can be declared `state` to persist across simulation steps – the standard way to hold onto a handle returned by external C/C++ code for the lifetime of the simulation:

```
1 state int* handle = create_motor(); // see Section 10.5, The Opaque
    Pointer Pattern
```

Note: arithmetic on pointers (e.g. `p + 1`) is not currently supported – use indexing (`p[i]`) to walk through memory instead.

5. Number Literals & Suffixes

Engineering suffixes are part of the number token — no space between digits and suffix. Also scientific e notation is supported (1e-6, 1e3).

Suffix	Multiplier	Example	Value
`p`	10^{-12}	`10p`	10 pF / ps
`n`	10^{-9}	`100n`	100 nH / ns
`u`	10^{-6}	`2u`	2 μF / μs
`m`	10^{-3}	`2m`	2 mH / ms
(none)	1	`400`	400
`k`	10^{3}	`20k`	20 000
`Meg`	10^{6}	`1Meg`	1 000 000
`G`	10^{9}	`2G`	2 000 000 000

```
R<1k>()[a, b];     // 1 000 Ohm
C<2u>()[node, gnd]; // 2e-6 F
L<2m>()[in, out]; // 2e-3 H
```

6. Operators

6.1 Arithmetic

Operator	Description
`+`	Addition
`-`	Subtraction
`*`	Multiplication
`/`	Division
`**`	Power
`%`	Modulo

6.2 Bitwise & Integer Operators

Bitwise operators manipulate the underlying binary representations of data.

Operator	Description
`&`	Bitwise AND
`|`	Bitwise OR
`^`	Bitwise XOR
`«`	Left shift
`»`	Right shift
`~`	Bitwise NOT (unary)

Important: Bitwise operators are strictly restricted to `int` and `unsigned int` types. Applying them to `double` or `float` variables will trigger a semantic error at compile time.

Note: `&` and `*` are overloaded by position. Between two operands they mean bitwise AND and multiplication respectively; in front of a single operand with nothing to their left, they instead mean address-of and dereference – see §4.5, Pointers.

6.3 Comparison (return 1.0 or 0.0)

Operator	Description
`==`	Equal
`!=`	Not equal
`>`	Greater than
`<`	Less than
`>=`	Greater than or equal
`<=`	Less than or equal

6.4 Logical

Operator	Description
`&&`	Logical AND
`||`	Logical OR
`!`	Logical NOT

6.5 Precedence (highest to lowest)

Operators	Description
`f()`, `a[]`	Function call, array subscript
`**`	Power (right-associative)
`-x`, `!x`, `~x`	Unary minus, logical NOT, bitwise NOT
`&x`, `*x`	Address-of, dereference (unary)
`*`, `/`, `%`	Multiplicative
`+`, `-`	Additive
`«`, `»`	Bitwise shift
`>`, `<`, `>=`, `<=`	Relational
`==`, `!=`	Equality
`&`	Bitwise AND
`^`	Bitwise XOR
`|`	Bitwise OR
`&&`	Logical AND
`||`	Logical OR

As in mathematics, `**` binds tighter than unary minus and groups from the right: `-x**2` is $-(x^2)$, `2**3**2` is $2^9 = 512$, and `2**-1` is 0.5. Write `(-x)**2` to square a negated value.

7. Variables & State

7.1 Local Variable

Re-initialised to the given value every simulation step.

```
double x = 0;
double a = 1.5, b = 2.5; // multiple declarations
```

7.2 State Variable

Initialised once at $t = 0$, then retains its value across timesteps. Equivalent to a register or flip-flop in hardware.

```
state double integrator = 0;  // starts at 0, keeps value each step
state double prev_error  = 0;
```

8. Control Flow

8.1 If / Else If / Else

```
if (condition) {
    // ...
} else if (other_condition) {
    // ...
} else {
    // ...
}

if (ref > carrier) {
    output = vdc;
} else {
    output = -vdc;
}
```

Note: circuit elements (`R`, `C`, sources, ...) and module instances may not appear inside an `if` or a `while`: the circuit's structure is fixed when the deck is compiled, and the compiler rejects them. To choose at run time, keep the element outside and switch what drives it — a source's value, or a conductance written as a `VCCS`.

8.2 While Loop

```
while (condition) {
    // ...
}

// Modulo via repeated subtraction
double T = 1.0 / freq;
double t = time;
while (t > T) {
    t = t - T;
}
```

9. Functions

Functions are called at runtime (not flattened). They can only accept and return logic types (`double`, `int`, `float`). Wires and physical primitives are not allowed as parameters.

9.1 Declaration

```
func <return_type> <name>(<type> <param>, ...) {
    // body
    return expression;
}

func double sineWave(double freq) {
    double pi = 3.14159265;
    double w  = 2.0 * pi * freq;
    return sin(w * time);
}
```

```
11
12 func double triangleWave(double freq) {
13     double T = 1.0 / freq;
14     double t = time;
15     while (t > T) {
16         t = t - T;
17     }
18     double result = 0;
19     if (t < T / 2.0) {
20         result = (4.0 * t / T) - 1.0;
21     } else {
22         result = 3.0 - (4.0 * t / T);
23     }
24     return result;
25 }
```

9.2 Call

```
1 double ref     = sineWave(50);
2 double carrier = triangleWave(20k);
```

10. Modules

Modules are the primary building block of a Hover program. They are *flattened* at compile time — there is no runtime module overhead, only the resulting nets and logic objects. Every module belongs to exactly one domain, declared by a prefix keyword. The domain controls what the module may contain and when it executes each timestep.

10.1 Domains

Domain	Allowed	Forbidden
`analog module`	Physical primitives, `V()`/`I()`, math, `if`, function calls	`state`, `while`
`digital module`	`state`, `if`, `while`, math, function calls	Physical primitives
`module`	Wire declarations, `V()`/`I()` reads, `state`, function calls, sub-module instantiation	Computation, `if`, `while`

Analog modules model continuous-time device physics. State lives in physical elements (capacitors, inductors) managed by the MNA engine — not in software variables. `if` is permitted for clamping and saturation detection; `while` is forbidden because it risks infinite loops inside the Newton-Raphson convergence loop. This makes `analog module` the mechanism for defining custom device models — transistors, diodes, transformers — that stamp directly into the MNA matrix alongside built-in primitives.

Digital modules model discrete-time and event-driven behaviour. `state` variables persist across timesteps, equivalent to registers. They may not instantiate hardware — the bridge between a

digital output and a physical primitive is always through a logic port of an analog module or a driven source in a structural module.

Structural modules (plain `module`) are wiring harnesses. They declare signal wires, read MNA node voltages, and instantiate sub-modules. They contain no computation.

10.2 Domain Restrictions

Feature	analog	digital	module
Physical primitives	√	–	√
`V()` / `I()`	√	√	√
Function calls	√	√	√
Math / binary expressions	√	√	–
`if` statement	√	√	–
`while` loop	–	√	–
`state` variables	–	√	–
Standalone assignment	√	√	–
Wire declarations	√	√	√
Signal declarations	√	√	√ *
Sub-module instantiation	√	√	√

* Structural module signal initializers are restricted — see below.

Structural Module Initializer Rules

Signal declarations inside a plain `module` are permitted but their initializer expression is restricted. Only literals, identifiers, and function calls are accepted. Binary arithmetic is not allowed — move computation into a `digital` or `analog` sub-module.

```
// Inside module (structural)
double x = 0;           // allowed -- literal
double x = other_sig;   // allowed -- identifier
double x = V(node);     // allowed -- function call
double x = signal();    // allowed -- function call

double x = -5;          // allowed -- negated literal
double x = a + b;       // ERROR -- binary expression
double x = 2.0 * freq; // ERROR -- binary expression
x = something;          // ERROR -- standalone assignment
```

10.3 Why Two Computational Domains

The separation between `analog` and `digital` reflects a fundamental difference in how each domain interacts with the circuit solver.

Analog modules are solved by the MNA engine. Their outputs — device currents, conductances — are stamped into the matrix and become part of the system $G \cdot x = b$. The solver iterates over analog logic inside its Newton-Raphson loop, recomputing device currents at each iteration until

the circuit converges. Analog logic must therefore be purely combinational: given the same node voltages, it must produce the same currents. A `state` variable inside an analog module would accumulate on every Newton-Raphson iteration rather than once per timestep, corrupting the solution.

Digital modules run outside the solver. They execute once per accepted timestep in Phase A, before the matrix is assembled. Their outputs are treated as fixed inputs to the MNA for that timestep — the solver never re-enters digital logic during Newton-Raphson iteration. This makes `state` variables safe: they update exactly once per timestep regardless of how many inner iterations the solver takes.

In short: analog logic is *inside* the solver loop, digital logic is *outside* it. Mixing the two — a `state` variable in an analog module, or a physical primitive in a digital module — would produce incorrect results, which is why the semantic analyser forbids it.

10.4 Bracket Convention

Brackets	Purpose
`< >`	Static parameters — compile-time constants
`( )`	Logic ports — runtime signals
`[ ]`	Physical ports — electrical wire connections

10.5 Declaration

```
[analog|digital] module Name<type param, ...>(direction type port,
    ...)
    [wire port, ...]
{
    // body
}
```

`direction` is `input` or `output`. Static parameters are evaluated at elaboration time and are available as constants inside the body.

```
// Analog module: device physics, no state
analog module NPN<double Is, double Bf, double Vt>()
    [wire base, wire collector, wire emitter]
{
    double vbe = V(base) - V(emitter);
    if (vbe > 0.75) { vbe = 0.75; }
    if (vbe < -5.0) { vbe = -5.0; }
    double i_be = (Is / Bf) * (2.71828 ** (vbe / Vt) - 1.0);
    current_source<>(i_be)[base, emitter];
}

// Digital module: discrete state, no hardware
digital module PI<>(input double meas, output double ctrl)[]
{
    state double integral = 0;
    double error = 3.0 - meas;
    if (dt > 0) {
        integral = integral + error * dt;
    }
    ctrl = 0.5 * error + 100.0 * integral;
}

```

```
// Structural module: wiring only
module main<>()[]
{
    wire motor_p;
    double ctrl_sig = 0;
    double measurement = V(motor_p);

    module pi_controller = PI<>(measurement, ctrl_sig)[];
    voltage_source<>(ctrl_sig)[motor_p, gnd];
    R<1k>()[motor_p, gnd];
}
```

10.6 Instantiation

```
module instance_name = ModuleName<static_args>(logic_args)[wire_args
    ];
```

```
module filter = LC_filter<2m, 2u>()[in, lc_out, gnd];
module ctrl   = PI<>(measurement, ctrl_sig)[];
```

`module instance_name =` is mandatory, except for physical primitives.

10.6.1 Named Parameters and Defaults

Static parameters may be given by name, and a declaration may give them defaults. Positional arguments come first, then `name=value`; each parameter is bound once, and an unbound parameter takes its default (an error if it has none). A default may use earlier parameters.

```
module RC<double R, double C = 1u, double tau = R * C>()[wire i, wire
    o] { ... }

module r1 = RC<1k>()[v, a];                // C = 1u
module r2 = RC<1k, C=2u>()[v, b];
module r3 = RC<C=10n, R=10k>()[v, c];
```

Named arguments are only valid inside `<...>`. Among the physical primitives only the independent sources take named parameters: `ac` and `ac_phase` (see `.ac`).

10.7 Execution Order

Each timestep, Phase A executes logic domains in a fixed causal order:

`module` ⟶ `digital` ⟶ `analog`

Structural modules read MNA state first, making voltages and currents available as signals. Digital modules then compute control outputs from those signals. Analog modules run last, computing device currents from the updated node voltages and control signals. Phase B then stamps all driven sources into the MNA matrix before the linear solve.

11. C/C++ Integration (FFI)

Because Hover transpiles circuit descriptions and logic into a high-performance C++ simulation environment, it provides native mechanisms for calling external C or C++ functions directly from your Hover source code. This is achieved using a combination of the `importc` directive and the `extern func` declaration.

11.1 The `importc` Directive

Unlike the standard `import` directive (which evaluates other Hover files), `importc` is a direct pass-through to the C++ code generator. It does not parse the target file. Instead, it instructs the compiler to inject a C++ `#include` statement at the top of the generated simulation code.

```
importc "my_custom_controller.hpp"; // Includes a local header file
importc "<cmath>";                  // Includes a standard C++
    library
```

This ensures that when the final C++ code is compiled, the external function definitions are available to the linker.

11.2 `extern func` Declarations

To call an external C/C++ function, Hover's semantic analyzer must first know its signature (parameters and return type) to enforce type safety during compilation. You declare this using the `extern func` keyword.

An `extern func` declaration acts as a forward declaration. It defines the interface without providing a body:

```
extern func double calculate_gain(double error, double dt, int mode);
extern func int check_status(unsigned int code);
```

11.3 Putting it Together

Once a C++ header is included via `importc` and the function signature is registered via `extern func`, the external function can be called anywhere standard functions are allowed (e.g., inside `digital` or `analog` modules).

```
// 1. Inject the C++ header into the generated output
importc "native_math.h"

// 2. Tell Hover's semantic analyzer about the function signature
extern func double fast_rsqrt(double value);

// 3. Use the external function inside a logic block
digital module RMS_Calculator<>(input double val, output double
    result)[]
{
    state double sum_squares = 0;

    // ... filtering logic ...

    // Call the native C++ function
    result = val * fast_rsqrt(sum_squares);
}
```

11.4 Handling Structs and Complex Types

Because Hover's type system is strictly limited to primitive numeric types (`double`, `float`, `int`, `unsigned`), it does not natively understand C/C++ `structs`, `classes`, or pointers. You cannot directly map a C++ struct as a parameter or return type in an `extern func` signature.

To interface with external C++ code that requires complex types, you must write a **primitive wrapper** on the C++ side.

For example, if you have a native C++ function taking a struct:

```
// C++ struct you want to use
struct MotorParams { double Ld, Lq, Flux; };
double compute_torque(MotorParams p, double id, double iq);
```

You create a "flattened" C++ wrapper function and import that wrapper into Hover:

```
// 1. C++ Wrapper (defined in motor_wrappers.hpp):
// double wrap_compute_torque(double Ld, double Lq, double Flux,
    double id, double iq) {
//     MotorParams p = {Ld, Lq, Flux};
//     return compute_torque(p, id, iq);
// }

// 2. Hover declaration:
importc "motor_wrappers.hpp"
extern func double wrap_compute_torque(double Ld, double Lq, double
    Flux, double id, double iq);
```

11.5 The Opaque Pointer Pattern

With Hover's support for pointers, the most robust way to manage persistent C/C++ structs is by using **opaque pointers**. You can instantiate a struct on the C++ side, cast its memory address to a primitive pointer type (such as `int*`), and pass that pointer back to Hover. Hover simply holds onto the pointer and passes it back to C++ when needed, without needing to know the struct's internal layout.

```
// 1. C++ Wrapper (motor_wrappers.hpp)
struct MotorParams { double Ld, Lq, Flux; };

// Create the struct and return its address as an int*
int* create_motor() {
    MotorParams* p = new MotorParams{0.002, 0.002, 0.15};
    return reinterpret_cast<int*>(p);
}

// Accept the int* from Hover, cast it back, and use it
double compute_torque(int* handle, double id, double iq) {
    MotorParams* p = reinterpret_cast<MotorParams*>(handle);
    return p->Flux * iq; // (simplified for example)
}
```

```
// 2. Hover Script
importc "motor_wrappers.hpp"

extern func int* create_motor();
extern func double compute_torque(int* handle, double id, double iq);

```

```
digital module MotorControl<>(input double id, input double iq,
    output double torque)[]
{
    // Store the opaque pointer as state so it persists across
        timesteps
    state int* motor_handle = create_motor();

    // Pass the pointer back to C++ for computation
    torque = compute_torque(motor_handle, id, iq);
}
```

Warning on Pointer Sizes: Always use true pointer types (like `int*`) to hold memory addresses. Do not attempt to cast or store a pointer inside a regular 32-bit `int` or `unsigned int` variable. Modern operating systems use 64-bit memory addresses; truncating a 64-bit pointer into a 32-bit integer will cause a segmentation fault when passed back to C++.

Note on Compilation: When building a simulation that relies on external FFI files, ensure that the corresponding `.c` or `.cpp` source files are included in your build system (e.g., added to the `Makefile`) so they are correctly compiled and linked with the generated Hover output.

12. Physical Primitives

Physical primitives are stamped directly into the MNA matrix. They use the same bracket convention as modules but do not need a name or `module` keyword.

```
<Primitive> [instance_name] < static_value > ( logic_signal ) [ wire1
    , wire2 ];
```

The instance name is optional. It is what lets the rest of the design *refer* to this particular element — see Section 12.1.

Primitive	Parameters	Ports	Description
`R`	`<resistance>`	`[n+, n-]`	Resistor
`L`	`<inductance>`	`[n+, n-]`	Inductor
`C`	`<capacitance>`	`[n+, n-]`	Capacitor
`voltage_source`	`<dc_value, ac=, ac_phase=>`	`[n+, n-]`	Ideal voltage source
`current_source`	`<dc_value, ac=, ac_phase=>`	`[n+, n-]`	Ideal current source
`VCCS`	`<gm>`	`[n+, n-, nc+, nc-]`	Voltage-controlled current source, $I = g_m \cdot V(nc+, nc-)$
`VCVS`	`<k>`	`[n+, n-, nc+, nc-]`	Voltage-controlled voltage source, $V = k \cdot V(nc+, nc-)$
`CCCS`	`<beta>`	`[n+, n-, sense]`	Current-controlled current source, $I = \beta \cdot I(sense)$
`CCVS`	`<r>`	`[n+, n-, sense]`	Current-controlled voltage source, $V = r \cdot I(sense)$

Note the asymmetry in the last four rows: the *voltage*-controlled pair takes **four wires** — two output terminals and two control terminals — because a control voltage is a property of a node pair. The *current*-controlled pair takes **three**: two output terminals and the **name of an element**, because a current in the MNA formulation is not a property of a node pair but a variable belonging to one specific branch.

12.1 Named Elements

Any primitive may be given an instance name, written between the primitive type and the `<>` list:

```
R       rsense<1m>()    [b, s];    // named
voltage_source vsense<0>() [mid, gnd];
R       <1k>()          [in, out];   // unnamed - still perfectly valid
```

A name is not decoration. It is the only way to *address* an element, and three features depend on it:

- `I(rsense)` — reading the element's current from logic (Section 13.1).
- `.save(rsense)` — logging that current to the CSV.
- the trailing `sense` entry of `CCCS`/`CCVS`.

Names scope to the module instance, exactly the way wires do. A module containing `voltage_source vsense<0>()[a, b];` instantiated twice as `a` and `b` yields two distinct elements, `main.a.vsense` and `main.b.vsense`. Within the module body you write the bare name (`I(vsense)`) and it resolves to the enclosing instance; from a top-level directive you write the full flattened path (`.save(main.a.vsense)`).

An unnamed primitive still gets an internal name of the form `main.R_3`, but you should never write one: the index shifts whenever a component is added above it. The compiler deliberately omits synthesized names from its "did you mean" suggestions for this reason.

Restrictions. An element name may not collide with another element in the same module, with a wire, or with a logic signal — all three are compile errors. It also may not be of the form `<Type>_<number>`, which is reserved for synthesized names.

12.2 Current-Controlled Sources (CCCS / CCVS)

`CCCS` and `CCVS` are controlled by the current through another element, named as the **third** `[]` entry:

```
CCCS<beta>() [out_p, out_n, sense_element];  // I_out = beta * I(
    sense)
CCVS<r>()    [out_p, out_n, sense_element];  // V_out = r    * I(
    sense)
```

The first two entries are ordinary wire terminals. The third is *not* a wire — it is the name of an element declared elsewhere in the design, and it must name an element that owns a branch row (Section 13.1). Supplying only two entries is a compile error rather than a silently dead stamp.

The reference may point **forward**. The sensed source is very often declared below the controlled source in the same body, and the compiler resolves all sense references in a post-pass once the whole design is flattened, so declaration order never matters.

12.2.1 The 0 V Ammeter Idiom

Because only branch-bearing elements have a current to sense, the standard technique — the same one SPICE uses — is to insert a **0 V voltage source** in series with the branch you care about. It is electrically invisible (zero volts across it) but it creates the branch row that makes the current a solved unknown:

```
// Measure the current through the 1k resistor, then mirror it 3:1.
R<1k>()                      [in, mid];
voltage_source vsense<0>() [mid, gnd];   // ammeter: I(vsense) == I
    through R
CCCS<3>()                    [gnd, out, vsense];
```

Pointing a `CCCS`/`CCVS` at a resistor, capacitor, `VCCS` or `current_source` instead is a compile error, and the diagnostic prints the ammeter declaration you need.

12.2.2 Terminal Conventions

Element	Meaning of `[n+, n-]`
`CCCS`	The generated current $\beta \cdot I(sense)$ flows *from* `n+`, through the source, *to* `n-`. It is therefore drawn out of the `n+` node and injected into the `n-` node.
`CCVS`	Sets $V(n+) - V(n-) = r \cdot I(sense)$. r has units of ohms (transresistance).

Self-reference is rejected. A `CCVS` that senses its own branch would produce a structurally singular matrix, so the compiler rejects it outright.

12.2.3 Worked Example

```
.tran(0, 100u, 1u);
.save(main.out, I(main.vsense), main.hn);

analog module main<>()[] {
    wire in, mid, out, hn;

    voltage_source vsupply<2>()  [in, gnd];  // 2 V supply
    R<1k>()                      [in, mid];  // -> 2 mA
    voltage_source vsense<0>()   [mid, gnd]; // ammeter, I(vsense) =
        2 mA

    // Current mirror: 3 x 2 mA = 6 mA injected into 'out'
    CCCS<3>()   [gnd, out, vsense];
    R<500>()    [out, gnd];                  // -> V(out) = 3.0 V

    // Transresistance stage: 100 ohm * 2 mA = 0.2 V
    CCVS hout<100>() [hn, gnd, vsense];
    R<1Meg>()        [hn, gnd];

    double i_measured = I(vsense);   // 2e-3
    double i_hout     = I(hout);     // CCVS has a branch too
}
```

This produces `main.out` = 3.0 V, `main.hn` = 0.2 V and `I(main.vsense)` = 2 mA, constant for the whole run.

Note: engineering suffixes are **case-sensitive** — the megohm suffix is `Meg`, and `1meg` is a syntax error. See Section 5.

12.3 Driven Sources (Logic-controlled)

Pass the controlling signal through the logic port `()`:

```
voltage_source<>(generator_output)[in, gnd]; // driven by
    generator_output each step
current_source<>(ctrl_signal)[n+, n-];
```

12.4 Fixed Sources (Compile-time value)

```
voltage_source<12>()[vdd, gnd]; // constant 12 V
R<1k>()[out, gnd];              // 1 kOhm to ground
C<100n>()[node, gnd];           // 100 nF
L<10u>()[a, b];                 // 10 uH
```

13. Built-in Intrinsics

These are available everywhere without import.

Function	Description
`V(wire_name)`	Voltage at MNA node from the previous timestep
`I(element_name)`	Current through a named branch-bearing element, from the previous timestep (Section 13.1)
`idt(x(t))`	integrates the value over time $\int_0^t x(\tau)d\tau$, only works in analog modules
`ddt(x(t))`	derives the value relative to time $dx(t)/dt$, only works in analog modules

```
double vOut  = V(lc_out);   // voltage on net lc_out
double iCoil = I(Vsense);   // current through Vsense branch
double wave  = sin(2.0 * 3.14159265 * 50 * time);
```

Note: `V()` and `I()` always return values from the *previous* MNA solve — they cannot read the result of the current timestep.

13.1 Reading Currents — `I()`

`I(element)` reads the current through a **named** circuit element (Section 12.1). Unlike `V()`, which takes a wire, its argument is an element instance name:

```
double iCoil = I(vsense);        // module-local name
```

Element references are module-local, exactly like `V()`'s nets: a bare name resolves to the element of that name in the same module instance. In top-level directives, where there is no enclosing instance, use the full flattened path: `.save(I(main.probe.vsense))`.

13.1.1 Which Elements Have a Current

Only elements that own their own row in the MNA matrix — an extra unknown for their current — can be read. For everything else the current is *derived* from node voltages rather than solved for, so there is no variable to point at.

Readable with `I()`	**Not readable**
`voltage_source`	`R`
`L`	`C`
`VCVS`	`VCCS`
`CCVS`	`CCCS`, `current_source`

This is the same set that may appear as a `CCCS`/`CCVS` sense reference, and the same set `.save()` accepts as a current column — the three features share one definition.

To measure a current through anything in the right-hand column, put a **0 V voltage source** in series with it as an ammeter and read that instead (Section 12.2):

```
R<1k>()                        [in, mid];
voltage_source rload_amp<0>() [mid, gnd];
double i = I(rload_amp);
```

A bad `I()` argument is a **hard compile error**, not a runtime warning — and the compiler distinguishes "no such element" from "that element has no branch current", because the fixes are different. The latter prints the ammeter declaration for you.

13.1.2 Sign Convention

`I()` is positive when current flows from the element's `n+` terminal, through the element, to its `n-` terminal — the passive sign convention, as in SPICE. A consequence worth internalising: **a source that is delivering power reads negative**, because its current flows internally from `n-` to `n+`.

```
voltage_source vsupply<2>() [in, gnd];   // 2 V supply feeding a 1k
    load
voltage_source vsense<0>()  [mid, gnd];  // ammeter in that same loop

// I(vsense)  = +2 mA   (current enters its n+ terminal)
// I(vsupply) = -2 mA   (same 2 mA, but the supply is sourcing it)
```

Note: `I()` reads the previous solve, and is redirected to the previous Newton iterate inside `nr_prev()` — see below.

13.2 nr_prev()

`nr_prev(expr)` evaluates `expr` using the node voltages and branch currents from the *previous Newton–Raphson iteration* of the current timestep's solve. It is the iteration-level counterpart to `state`: where `state` remembers a value across *timesteps*, `nr_prev()` remembers it across *iterations* within a single timestep's nonlinear solve.

Mechanically, for `V()` and `I()` `nr_prev()` introduces no storage of its own. It re-emits its argument with every nested `V()` and `I()` call routed to its previous-iteration source (`api_V_prev` / `api_I_prev`) instead of the working-iterate source (`api_V` / `api_I`). Parameters, arithmetic, `time` and `dt` are emitted unchanged. A variable assigned by analog code is read from a per-instance slot holding its value at the previous iterate — see "`nr_prev()` of a variable" below. Nesting

is well defined: `nr_prev()` inside an otherwise ordinary expression, or even `nr_prev()` inside `nr_prev()`, behaves correctly and does not leak the redirection to sibling expressions.

Construct	Value returned
`V(a)`, `I(e)`	most recently solved value (the iterate the solver is currently refining)
`nr_prev(V(a))`	value of `V(a)` one NR iteration earlier, same timestep
`state double x`	value of `x` carried over from the previous timestep

The intended use is writing iteration-aware numerical aids — junction limiting, damping, source ramping — directly in the language rather than hard-coding them into the solver. Because the solver is general (it makes no assumption that a node is a semiconductor junction), the per-device limiting that keeps a stiff exponential from overflowing belongs in the model. `nr_prev()` supplies the "last guess" that such a limiter needs to bound how far the next guess may move.

```
// Junction limiting expressed in-language: bound the per-iteration
// change in junction voltage using last iteration's value.
double vNew = V(p) - V(n);           // current working iterate
double vOld = nr_prev(V(p) - V(n)); // same junction, previous NR
    iterate

double dv = vNew - vOld;
if (dv >  vt) { dv =  vt; }          // clamp the step, not the
    voltage
if (dv < -vt) { dv = -vt; }
double vLim = vOld + dv;             // limited voltage feeds the
    device model
```

Note: `nr_prev()` is iteration history, not a general time-domain memory. To remember a value from a previous *timestep*, use `state`. On the first iteration of a timestep's solve there is no earlier iterate, so `nr_prev()` returns that step's starting guess — meaning the very first iteration applies no limiting, which is the intended cold-start behavior.

13.2.1 nr_prev() — Supported Forms

`nr_prev(expr)` accepts *any* expression. There is no special argument grammar: the compiler emits the full translation of `expr` and redirects every nested `V()` and `I()` leaf to its previous-iteration source (`api_V_prev` / `api_I_prev`). Everything else in the expression — parameters, constants, `time`, `dt`, arithmetic, `sin`/`cos` — is emitted unchanged. The redirection is purely lexical over the `V()`/`I()` leaves, so it composes with arbitrary surrounding math.

Form	Emitted (conceptually)	Meaning
`nr_prev(V(a))`	`api_V_prev(a)`	node voltage, previous NR iterate
`nr_prev(I(e))`	`api_I_prev(e)`	branch current, previous NR iterate
`nr_prev(V(a) - V(b))`	`api_V_prev(a) - api_V_prev(b)`	differential voltage, previous iterate
`nr_prev(V(a)) - nr_prev(V(b))`	`api_V_prev(a) - api_V_prev(b)`	identical to the row above
`nr_prev(V(a) / vt)`	`api_V_prev(a) / vt`	`vt` (param) passes through unchanged
`nr_prev(sin(V(a)))`	`sin(api_V_prev(a))`	redirection reaches into call arguments
`nr_prev(nr_prev(V(a)))`	`api_V_prev(a)`	nesting is idempotent (already redirected)

```
double vbe   = nr_prev(V(b) - V(e));        // previous-iterate junction voltage
double vbe2  = nr_prev(V(b)) - nr_prev(V(e)); // exactly the same code
double iPrev = nr_prev(I(Vsense));          // previous-iterate branch current
double scaled= nr_prev(V(a) * 2.0 - V(b));  // params/constants pass through
```

Note: Parameters, constants, `time` and `dt` pass through unchanged, and so do inputs that no analog code writes (logic is sample-and-hold: constant across the iterations of a step). `nr_prev(time)`, `nr_prev(dt)`, or any expression containing no `V()`/`I()` leaves and no analog variable reduces to the plain expression. A *variable* inside `nr_prev()` is the subject of the next subsection.

13.2.2 nr_prev() of a variable — the limiter's own anchor

A variable that an analog module assigns, inside `nr_prev()`, means that variable as the evaluation at the previous Newton iterate left it: its last assigned value in that evaluation. This is what a junction limiter should be anchored on — the voltage the device itself used last iteration, after limiting, which is what SPICE stores per device (`DIOload` limits against `CKTstate0`, not against the node voltages). The variable may appear inside its own declaration:

```
double v  = V(a) - V(k);
double vl = pnjlim(V(a) - V(k), nr_prev(vl), nvt, vcrit); // anchored on itself
if (vl > 60.0 * nvt) { vl = 60.0 * nvt; }                 // nr_prev(vl) sees this too
double vd = nodiff(vl);
```

`nr_prev(V(a) - V(k))` would anchor the same limiter on the previous *node* voltages instead. The two differ exactly when it matters: when the iterate stops moving while the junction is still far from it, a node anchor sees a step under $2\,nV_t$ and lets the junction jump all the way (to the model's $60\,nV_t$ hold, from where Newton walks down about one nV_t per iteration); a device anchor keeps compressing the step. A diode behind $1\,\Omega$ stepped to $10\,\mathrm{V}$ took 45–48 iterations per plain-Newton retry with the node anchor and 6–7 with the device anchor.

The rules:

- **One slot per variable per module instance**, like `ddt()`: two diodes never share a value. The slot is written after every assignment to the variable in the analog phase, so it holds the variable's *last* value of that evaluation (above: after the hold).
- **The previous iterate's evaluation, never a probe.** The analog block runs many times per iterate — residual, Jacobian, trust-region trials, the convergence test. Only the evaluation at the iterate itself counts; the runtime tells it from the probes by the solution vector it is looking at (`runtime/vm/nr_locals.hpp`). A rejected step, truncation-error rejection or `.zcd` probe rolls the slots back with the rest of the state.
- **First iteration of a step**: the value from the last iterate of the previous accepted step (SPICE's accepted state). **Very first evaluation of the run** (operating point start): 0, the same as `nr_prev(V(...))` reads from the zero solution.
- **When the previous iterate is the current point** (the convergence test's "limiters released" evaluation, where `nr_prev(V(a))` equals `V(a)`): the value from the evaluation just before, at the same point — for a limiter, one more application of it. A limiter that is idle stays where it is; one still moving its variable shows up as a difference, and convergence is refused.
- **No derivative**: under automatic differentiation the value is a constant, as it is a stored number.
- **Analog module bodies only.** Inside a function it is a compile error — one C function serves every caller, so a function local has no instance to keep a value for; pass `nr_prev(x)` into the function from the module, as the models pass it into `pnjlim`. In a digital module it is an error too: logic runs once per step, outside Newton.

The standard library's junction models (`Diode`, both BJTs, `LED`, the MOSFET bulk junctions) and the MOSFET's `fetlim`/`limvds` are anchored this way.

Note: `nr_prev` reaches into *call arguments* (e.g. `sin(V(a))`), but not into a user function's *body*. If a function internally read `V()` it would not be redirected — though in practice functions cannot take `wire` arguments, so this case does not arise for node reads.

13.3 nodiff()

`nodiff(expr)` is the value of `expr`, but the Newton Jacobian treats it as a constant: its derivative is zero. It is a builtin like `nr_prev()`, needs no import, and is meant for the second half of junction limiting.

A limiter such as `pnjlim` returns a voltage near the previous iterate. If that voltage goes straight into the device equation, the Jacobian differentiates *through* the limiter, whose slope far from the previous iterate is tiny — the device then looks like an open circuit to Newton exactly while it is being limited. SPICE avoids this by linearising the device at the limited voltage and applying that tangent line to the *true* voltage (the companion model); `nodiff()` expresses the same thing in the language:

```
1 double v  = V(a) - V(k);
2 double vd = nodiff(pnjlim(V(a) - V(k), nr_prev(vd), nvt, vcrit));
3 double e  = jexp(vd, nvt);
4 double id = Is * (e - 1.0) + Is * e / nvt * (v - vd);   // f(vd) + f
      '(vd) (v - vd)
```

At a solution the limiter is idle, `vd` equals `v`, and `id` is the diode equation exactly. `<optoelectronics/leds>`' `LED` is written this way. Under the numerical (finite-difference) Jacobian, used when a deck cannot be differentiated automatically, `nodiff()` is a plain identity.

14. Special Keywords

Keyword	Description
`time`	Simulation time in seconds. Read-only, available everywhere. In analog code it is the time point the step solves for ($t_n + dt$, where SPICE evaluates its sources); in digital and structural logic it is the step's start t_n — logic runs once there and holds for the step. On a step that ends exactly on a time breakpoint (Section 3.4) analog code sees the instant just before it.
`gnd`	Ground node (0 V reference). Available everywhere as a wire.
`state`	Variable modifier — retains value across timesteps.
`dt`	Timestep in seconds. Read-only, available everywhere. In analog code, the step being solved; in a clocked digital module, the clock period.
`input/output`	Logic port direction in module declarations.

Because `time` and `dt` are read-only, declaring a variable with either name (`double dt = 1u;`) or assigning to one (`dt = 1u;`) is a compile error. A function parameter may still use the name; inside that function it refers to the argument.

15. Simulation Run Order

Each timestep executes in three phases, repeated until `time` > `stop`.

Phase A **Execute logic — three domain passes in fixed order**

1. ***module*** Structural logic runs first. Wire signals are read from the previous MNA solution via `V()` and `I()` and made available to downstream domains.
2. ***digital*** Digital logic runs second. `state` variables are updated, `if`/`while` blocks are evaluated, and control outputs are written to their signal names.
3. ***analog*** Analog device models run last. Node voltages from the previous solve and control signals from the digital pass are read to compute device currents and physics outputs.

Phase B **Stamp logic outputs into MNA**

Every driven `voltage_source` and `current_source` reads its controlling signal from the values written in Phase A and updates the MNA right-hand side vector b.

Phase C **Solve and advance**

The linear system $G \cdot x = b$ is solved (or iterated to convergence for implicit solvers). Node voltages and branch currents are committed. Signals listed in `.save` are recorded, and `time` advances by `dt`.

Repeat until `time > stop`.

The causal ordering within Phase A is fixed regardless of declaration order in the source file. A digital controller will always see the structural `V()` reads from the current step, and an analog device model will always see the digital control output from the current step.

16. Hovercraft — Library Mode

By default the compiler produces a **one-shot binary**: it runs the whole transient from `start` to `stop`, writes `simulation_output.csv`, and exits. Everything about the run is fixed at compile time.

Hovercraft mode compiles the same `.hvr` source into a **reusable shared library** instead — `libhovercraft.so` on Linux, `hovercraft.dll` on Windows — exporting a plain C ABI. A host program loads it and drives the simulation interactively: advance it a little, read a result, change an input based on what it read, advance again. That closed loop is the thing a one-shot binary structurally cannot do, and it is what the mode exists for — hardware-in-the-loop style controllers, parameter sweeps without recompiling, optimisation, and co-simulation with a host-side model.

```
./hover circuit.hvr --hovercraft
./hover circuit.hvr --hovercraft -o motor    // custom artifact name
```

Every exported symbol is prefixed `HVR_` and declared `extern "C"`; no C++ type crosses the boundary. The full ABI is declared in `runtime/hovercraft.h`, which is the authoritative reference — this section describes the model.

16.1 Compiler Options

Option	Effect
`-hovercraft`	Emit the shared library instead of a simulation binary. Nothing is executed after the build.
`-o <path>`	Write the artifact to `<path>` instead of the default name. `-o=<path>` is also accepted. Names the final artifact only — the generated `sim.cpp` always lands in the working directory, where it stays inspectable.
`-dump-ast`	Print the entry file's AST and exit.

No platform extension is appended to `-o`'s argument; the path is used verbatim. Flag order does not matter, and an unrecognised flag is an error rather than being silently ignored.

16.2 What Becomes Part of the ABI

The library's interface is derived from `main`'s own signature and the `.save()` directives. Three groups of symbols are generated, named after the things they expose:

Group	Comes from	Generated symbol
Params	`main`'s `<>` static args	`int HVR_set_param_<name>(double)`
Inputs	`main`'s `()` logic args	`void HVR_set_input_<name>(...)`
Outputs	every `.save()`d column	`double HVR_get_output_<name>(void)`

For a module declared

```
.save(main.vout, main.drive, main.vsense);
```

```
analog module main<double gain>(double vin, double[3] taps) [] { ... }
```

the library exports `HVR_set_param_gain`, `HVR_set_input_vin`, `HVR_set_input_taps`, `HVR_get_output_main_vout`, `HVR_get_output_main_drive` and `HVR_get_output_I_main_vsense`.

Non-identifier characters in a column name become underscores, which is why the branch-current column `I(main.vsense)` yields `HVR_get_output_I_main_vsense`. Should two columns ever mangle to the same symbol, the collision is resolved by omitting the per-signal getter; read those columns through `HVR_get_output()` by exact name instead.

16.2.1 Params vs. Inputs — an Important Distinction

	<> params	**() inputs**
Settable	only while `time == 0`	at any time, mid-simulation
Otherwise	returns `HVR_ERR_TIME`	—
To change	call `HVR_reset_sim()` first	just call the setter
Types	scalar `double`	scalar or array

A `<>` param is a *structural* quantity: it is folded into the design during elaboration, so it can only be re-read by starting the run over. A `()` input is a *signal*, read fresh by the logic on every timestep — that is the one you drive in a control loop.

Array inputs take an explicit length, since a C pointer carries none:

```
void HVR_set_input_vin(double v);                          // scalar
void HVR_set_input_taps(const double *values, long n);     // double[3]
```

Passing fewer elements than the array holds writes only the leading ones and leaves the rest untouched; the setter never reads past the array's declared size.

16.3 Driving the Simulation

Function	**Effect**
`int HVR_step(long n)`	Advance `n` minimum timesteps ($n \cdot dt$ seconds)
`int HVR_run(double secs)`	Advance an arbitrary duration in simulation seconds
`double HVR_get_time(void)`	Current simulation time. Reading it never starts the run
`int HVR_reset_sim(void)`	State → initial values, `time` → 0, OP re-solved if enabled. The log is **not** cleared
`void HVR_reset_log(void)`	Clear the log. Simulation state untouched

The simulation is initialised lazily, on the first call that actually needs state to exist. That is deliberate: it means a host may call every `HVR_set_input_*` and `HVR_set_param_*` it likes *before* the first solve ever happens, so the very first timestep already sees the host's values rather than the compiled-in defaults.

The `stop` time given to `.tran` does not bound a hovercraft run — the host decides when to stop by simply not calling `HVR_run` again. It survives in the manifest as `end_time`, a suggested default.

16.4 Reading Results

Two channels, for two different access patterns.

Per-signal reads — no allocation, intended for a host polling one or two signals every step:

```
double HVR_get_output_main_vout(void);        // zero-overhead, name
    fixed at compile time
int    HVR_get_output(const char *name, double *out);  // by exact
    column name
```

These return the signal's **current** value — the live node voltage, branch current, or logic value — not a logged row. `HVR_get_output()` returns `HVR_ERR_UNKNOWN` and leaves `*out` untouched for an unknown name, so "no such signal" stays distinguishable from a signal whose value genuinely is 0.

Bulk log queries — for waveforms:

```
HVRLogResult HVR_get_log(void);                                  //
    everything
HVRLogResult HVR_get_log_range(double t_start, double t_end);
HVRLogResult HVR_get_log_latest(void);                           // since
    the last run/step
HVRLogResult HVR_get_last_step(void);                            // one row
void         HVR_clear_log_before(double t);                     // bound
    memory
int          HVR_save_log(const char *filename);                 // opt-in
    CSV dump
```

```
typedef struct {
    double  *time;     /* n_rows entries                          */
    double **columns;  /* n_cols arrays, each n_rows entries      */
    char   **names;    /* n_cols column names                      */
    long     n_rows;
    long     n_cols;
} HVRLogResult;
```

Every `HVRLogResult` must be released with `HVR_free_log_result()`. The arrays are allocated on the C++ side and a caller in another language has no way to free them correctly itself.

Note that CSV export is **opt-in** here. A hovercraft build writes no file unless the host asks for one — unlike the standalone binary, which always produces `simulation_output.csv`.

16.5 Return Codes

Constant	Value	Meaning
`HVR_OK`	`0`	Success
`HVR_ERR_TIME`	`-1`	`HVR_set_param_*` called with `time != 0`
`HVR_ERR_IO`	`-2`	`HVR_save_log` could not open the file
`HVR_ERR_UNKNOWN`	`-3`	Invalid argument — negative duration, unknown signal name

16.6 Self-Description — `HVR_manifest()`

The per-project symbols above depend on the `.hvr` source, so their names cannot be declared in a shared header. Rather than requiring the host to keep the original source alongside the library, every hovercraft build embeds a JSON description of its own ABI:

```
const char *HVR_manifest(void);   // static storage, do not free
```

```
{
  "hovercraft_abi": 1,
  "time_step": 1e-05, "end_time": 0.01,
  "params":  [{"name": "gain", "symbol": "HVR_set_param_gain", "
     default": 0}],
  "inputs":  [{"name": "taps", "symbol": "HVR_set_input_taps",
               "kind": "array",  "ctype": "double", "length": 3},
              {"name": "vin",  "symbol": "HVR_set_input_vin",
               "kind": "scalar", "ctype": "double", "length": 1}],
  "outputs": [{"column": "main.vout", "symbol": "
     HVR_get_output_main_vout",
               "kind": "node"},
              {"column": "I(main.vsense)",
               "symbol": "HVR_get_output_I_main_vsense", "kind": "
                  current"}]
}
```

This makes a `.so` genuinely self-contained. It matters most for inputs: an exported symbol name carries no signature, and `HVR_set_input_<name>` is a one-argument scalar setter for a scalar input but a two-argument `(const T *values, long n)` setter for an array one. Guessing wrong does not fail cleanly — it segfaults — so a host should always dispatch on the manifest's `kind` field rather than on the symbol's existence.

16.7 Instance Model

A hovercraft library holds its entire simulation — state variables, the MNA system, the log — in **file-scope statics**. One loaded library is therefore exactly **one** simulation, and the `HVR_*` functions are not reentrant.

This has a consequence worth stating plainly, because the failure is silent: loading the same library file twice in one process does **not** give two independent simulations. `dlopen` returns the same handle for a path already loaded, so both references share one set of statics and writes through one show up in the other. To run several instances concurrently, give each one a **distinct copy of the library file**; a distinct path gets its own mapping with its own statics.

16.8 Caveat — What a <> Param Can Reach

A `<>` param is runtime-settable only where it is read by **equations in `main`'s own body**. If `main` instead passes it to a submodule (`Src<amp, freq>()`) or uses it as an element value (`R<rload>()`), elaboration folds it to a constant and stamps it into the netlist once — the setter will return `HVR_OK` and change nothing.

Practically, a design with tunable params wants `main` to be an `analog` module, since a structural `module main` cannot contain equations at all (Section 10.1) and therefore cannot read a `<>` param anywhere:

```
.save(main.vout, main.drive, main.vsense);
.tran(0, 10m, 10u, 10u);

// analog, not structural: it holds both the equation reading `gain`
// and the circuit elements that equation drives.
analog module main<double gain>(double vin, double[3] taps) []
{
```

```
    double drive = gain * vin + taps[0] + taps[1] + taps[2];

    wire vin_node, vout;
    voltage_source vsense<>(drive) [vin_node, gnd];
    R<1k>()   [vin_node, vout];
    C<100n>() [vout, gnd];
}
```

17. Full Example

Sinusoidal PWM inverter with LC output filter, demonstrating all three module domains working together.

```
.save(main.lc_out, main.generator_output);
.tran(0, 40m, 1u, 10u);
.solver(tr_bdf2);

//-- Helper functions ------------------------------------------

func double sineWave(double freq) {
    double pi = 3.14159265;
    return sin(2.0 * pi * freq * time);
}

func double triangleWave(double freq) {
    double T = 1.0 / freq;
    double t = time;
    while (t > T) {
        t = t - T;
    }
    if (t < T / 2.0) {
        return (4.0 * t / T) - 1.0;
    } else {
        return 3.0 - (4.0 * t / T);
    }
}

//-- Analog module: purely physical, state lives in L and C ----

analog module LC_filter<double inductance, double capacitance>()
    [wire in, wire out, wire ground]
{
    L<inductance>()[in, out];
    C<capacitance>()[out, ground];
}

//-- Digital module: discrete logic, no hardware ---------------

digital module PWM_generator<double freq, double carrier, double vdc>
    (output double target_voltage)[]
{
    double ref = sineWave(freq);
    double tri = triangleWave(carrier);
    if (ref > tri) {
        target_voltage = vdc;
    } else {
```

```
44          target_voltage = -vdc;
45      }
46  }
47
48  //-- Structural module: wiring only ----------------------------
49
50  module main<>()[]
51  {
52      wire in, lc_out;
53
54      // Signal wire: receives PWM output from digital module
55      double generator_output = 0;
56
57      // LC filter: L=2mH, C=2uF -> f0 ~2.5 kHz
58      module filter = LC_filter<2m, 2u>()[in, lc_out, gnd];
59
60      // PWM generator: 50 Hz output, 20 kHz switching, 400 V DC bus
61      module generator = PWM_generator<50, 20k, 400>(generator_output)
            [];
62
63      // Voltage source driven by digital module output each timestep
64      voltage_source<>(generator_output)[in, gnd];
65
66      // 1 kOhm load
67      R<1k>()[lc_out, gnd];
68  }
```

Domain Responsibilities

Domain	Role in this circuit
`analog module`	Stamps L and C into the MNA matrix. The inductor current and capacitor voltage are tracked by the solver automatically.
`digital module`	Compares sine reference against triangle carrier each step and writes ±400 V to `generator_output`.
`module`	Wires the filter and generator together. Routes `generator_output` to the driven voltage source.

Expected Output

Signal	Description
`main.lc_out`	≈ 400 V peak sine at 50 Hz
`main.generator_output`	Square wave ±400 V at 20 kHz

The LC filter passes 50 Hz (below the 2.5 kHz cutoff) and rejects the 20 kHz switching frequency (8× above cutoff, ≈64× attenuation).